INDEX

NAME: S. Kaviya ROLL NO: .....

SUBJECT: ..... STD: ..... DIV: .....

SCHOOL: .....

SR.NO.	DATE	TITLE	PAGE NO.	SIGNATURE
1	21/7	Bresenham's algorithm for Straight line		
2	21/7	Bresenham's algorithm for Circle.		
3	21/7	Bresenham's algorithm for ellipse		
4.	11/8	implementation of DDA algorithm.		
5.	11/8	Drawing object using Bresenham algorithm.		
6.	25/8	Rectangle using Bresenham		
7.	25/8	Traffic signal using Bresenham algorithm.		
8.	25/8	Smiley using Bresenham algo.		
9.	25/8	Olympic logo using Bresenham		
10.	22/9	Scan conversion of polygon using Floodfill		
11.	22/9	2D transformations in GL.		

SR.NO.	DATE	TITLE	PAGE NO.	SIGNATURE
12	29/9/26	2D Transformations on ☐		
13)	29/9/26	clipping on polygons.		
14)	6/10/26	Tranformations on cube		
15)	6/10/26	Transformations on Pyramid		
16		create a Multimedia app using audio/video		
17)		Capture video/audio from webcam		
18)		create a application to send/receive multimedia msg.		
19)		create a 3D blender & apply basic shading.		

Completed
12/10/26

2.1.4

Bresenham's algorithm

AIM:

To plot a straight line using Bresenham's algorithm.

Program:

```
import matplotlib.pyplot as PH

def bresenham (x1, y1, x2, y2):

    dx = x2 - x1
    dy = y2 - y1
    p = 2* dy - dx

    x = x1
    y = y1

    x = [x]
    y = [y]

    while x < x2:
        x += 1
        if p < 0:
            PH = 2* dy
        else:
            y += 1
            p = 2* dy - 2* dx
        x.append(x)
        y.append(y)

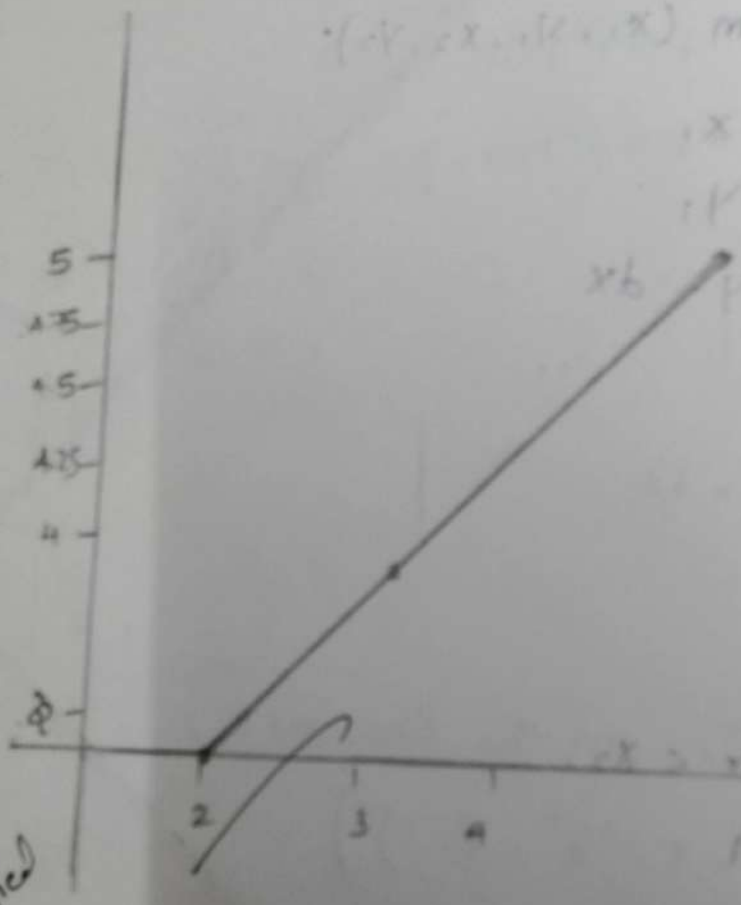
    return x, y
```

```
# input pts
x1, y1 = 2, 3
x2, y2 = 4, 5
x, y = bresenham (x1, y1, x2, y2)
plt.plot (x, y, 'b-', linewidth=2)
plt.scatter(x, y, color='red')
plt.grid (True)
plt.axis ('equal')
plt.show()
```

Result:

Using Bresenham algorithm the straight line has successfully plotted.

Program OUTPUT:-



O/P verified

Bresenham's algorithm for straight line (m<1).

21/7/26

Bresenham's algorithm for circle

AIM:

To Plot a circle using Bresenham's algorithm.

Program:

```
import matplotlib.pyplot as plt

def midpoint_circle(r):
x=0
y=r
p=1-r
x= []
Y= []
while x<=y:
    points: [
        (x,y), (y,x),
        (-x,y), (-y,x),
        (-x,-y), (-y,-x),
        (x,-y), (y,-x)
    ]
    for px, py in points
        x.append(px)
        Y.append(py)
    x+=1
    if p<0:
        p=p+2*x+1
```

```
        else:
            y-=1
            p=p+2*x+1-2*y

    return x, y

r=10

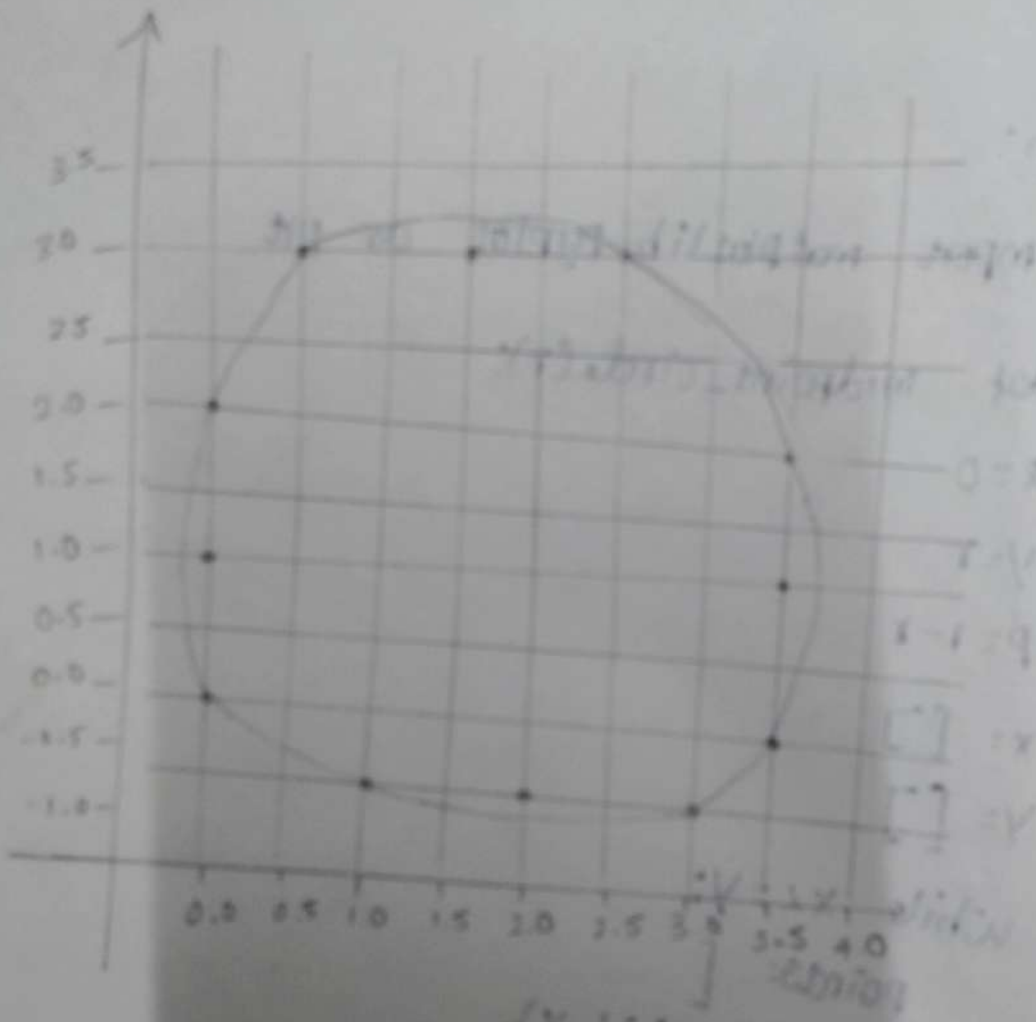
x, y= midPoint_Circle (r)

theta = np.linspace(0, 2*np.pi, 500)
xc = r*np.cos(theta)
```

28/7/24

Result:

Using Bresenham's algorithm the circle has successfully plotted.



Bresenham's algorithm for circle

2.17

Bresenham's algorithm for ellipse

AIM:

To plot an ellipse using Bresenham's algorithm.

Program:

```
import matplotlib.pyplot as plt
x = 0
y = ry
rx2 = rx * rx
ry2 = ry * ry
P1 = ry2 - (rx2 * ry) + (0.25 * rx2)
points = []
while (2*ry2*x) < (2*rx2*y):
    points.append (x, y)
    if P1 < 0:
        x = x+1
        P1 = P1 + (2*ry2*x) + ry2
    else:
        x = x+1
        y = y-1
        P = P1 + (2*ry2*x) - (2*rx2*y) + ry2
P2 = (ry2 * (x+0.5) * (x+0.5)) + (rx2 * (y-1) * (y-1)
while y >= 0
    points.append ((x,y))
```

```
if P2 >0:
    y = y-1
    P2 = P2 - (2*rx2*y) + rx2

else:
    x = x+1
    y = y-1
    P2 = P2 + (2*ry2*x) - \(2*2x2*y) + rx2

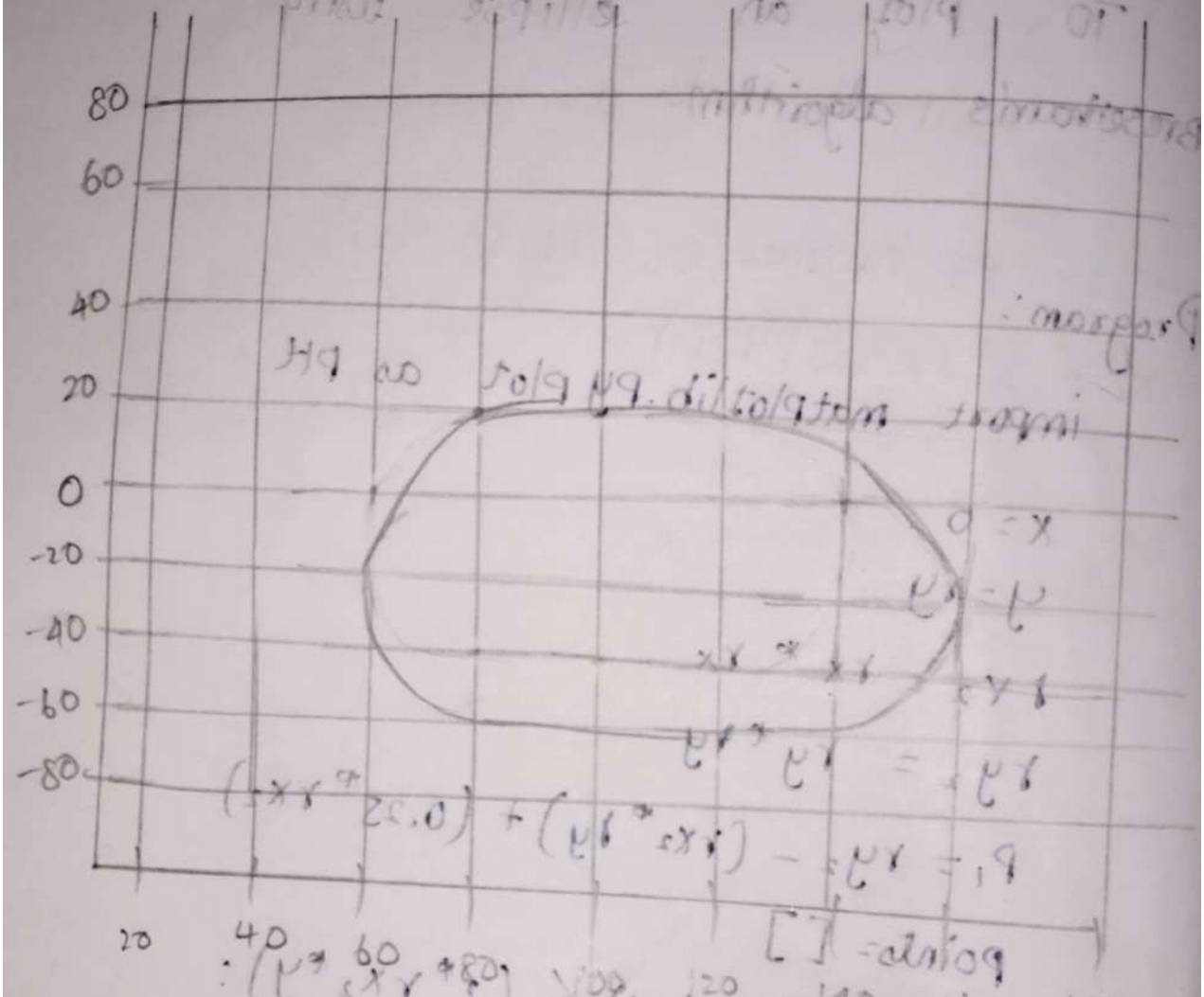
ellipse_points = []
for x,y in points:
    ellipse_points.append ((xc + x, yc+y))
    ellipse_points.append ((xc - x, yc+y))
    ellipse_points.append ((xc + x, yc-y))
    ellipse_points.append ((xc - x, yc-y))
return ellipse_points

xc = 0
yc = 0
rx = 100
ry = 50
plt.scatter (x_values, y_values, s=z)
plt.xlabel ("x_axis")
plt.ylabel ("y_axis")
plt.title ("Bresenham's ellipse Drawing Algorithm")
plt.axis ("equal")
plt.grid()
plt.show()
```

11/8/26

Result:

Therefore an ellipse has been drawn successfully.


80
60
40
20
0
-20
-40
-60
-80
20
40
60
80
100
120
140
160
180

Implementation of DDA algorithm

AIM:

To write a python code to implement DDA algorithm.

Program:

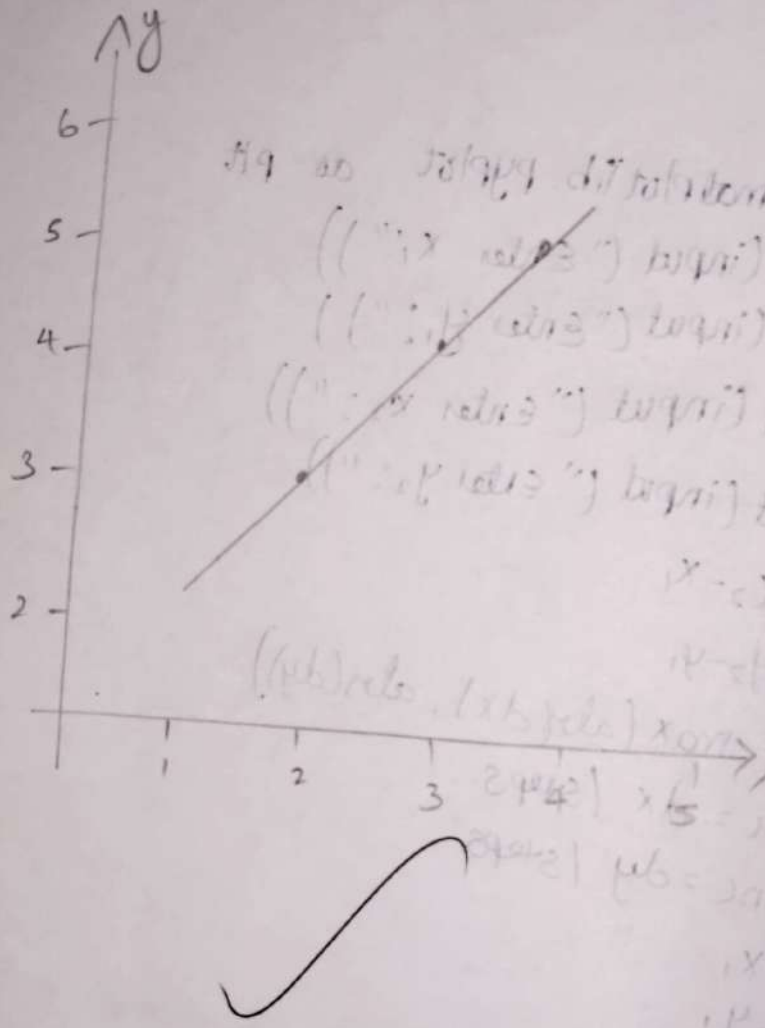
```
import matplotlib.pyplot as plt
x1 = int(input("Enter x1:"))
y1 = int(input("Enter y1:"))
x2 = int(input("Enter x2:"))
y2 = int(input("Enter y2:"))
dx = x2-x1
dy = y2-y1
steps = max(abs(dx), abs(dy))
x_inc = dx/steps
y_inc = dy/steps
x = x1
y = y1
x_points = []
y_points = []
for i in in range(steps+1):
    x_points.append(round(x))
    y_points.append(round(y))
    x+=x_inc
    y+=y_inc
```

```
plt.plot(x_points, y_points, marker='o')
plt.xlabel("x-axis")
plt.ylabel("y-axis")
plt.grid(True)
plt.show()
```

11/8/26

Result:

The straight line has been drawn using DDA algorithm.


y
6
5
4
3
2
1
2
3
x

1118

Object using Bresenham's algorithm.

AIM:

To draw an object using bresenham's algorithm.

Program:

```
import matplotlib.pyplot as plt
def bresenham's (x1, y1, x2, y2):
    points = []
    dx, dy = abs(x2 - x1), abs(y2-y1)
    sx = 1 if x1 < x2 else -1
    sy = 1 if y1 < y2 else -1
    err = dx - dy
    while True:
        points.append ((x1, y1))
        if x1 == x2 and y1 == y2:
            break
        e = 2 * err
        if e > -dy:
            err = dy
        if e < dx:
            err += dx
            y1 += sy
    return points.
plt.plot (* zip (* bresenham(20, 10, 20, 90))
flag = [(20, 85), (70, 85), (70, 55), (20, 55)]
```

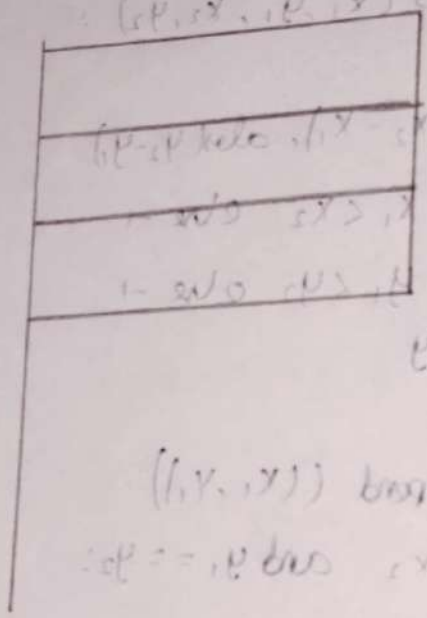
```
plt.fill([20, 70, 70, 20], [85, 85, 55, 55],
                                color="orange")
plt.fill([20, 70, 70, 20], [75, 75, 65, 65],
                                color="white")

plt.xlim(0, 100)
plt.ylim(0, 100)
plt.axis("off")
plt.show()
```

11/8/26

Result:

The object has been drawn successfully.



Flag using Bresenham algorithm

25/8

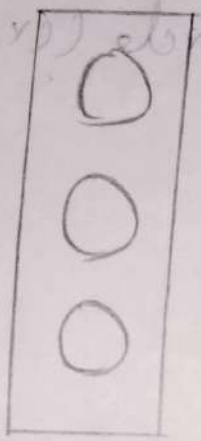
Traffic signal using Bresenham's Algorithm

AIM:

To draw a traffic signal using Bresenham's algorithm.

Code:

```
def bresenham_circle(cx, cy, r, color):
    x = 0
    y = r
    d = 3 - 2 * r
    while x <= y:
        for i in range(cx - x, cx + x + 1):
            plt.plot(i, cy + y, '.', color = color)
            plt.plot(i, cy - y, '.', color = color)
        for i in range(cx - y, cx + y + 1):
            plt.plot(i, cy + x, '.', color = color)
            plt.plot(i, cy - x, '.', color = color)
        if d < 0:
            d = d + 4 * x + 6
        else:
            d = d + 4 * (x - y) + 10
            y -= 1
        x += 1
```



25/8

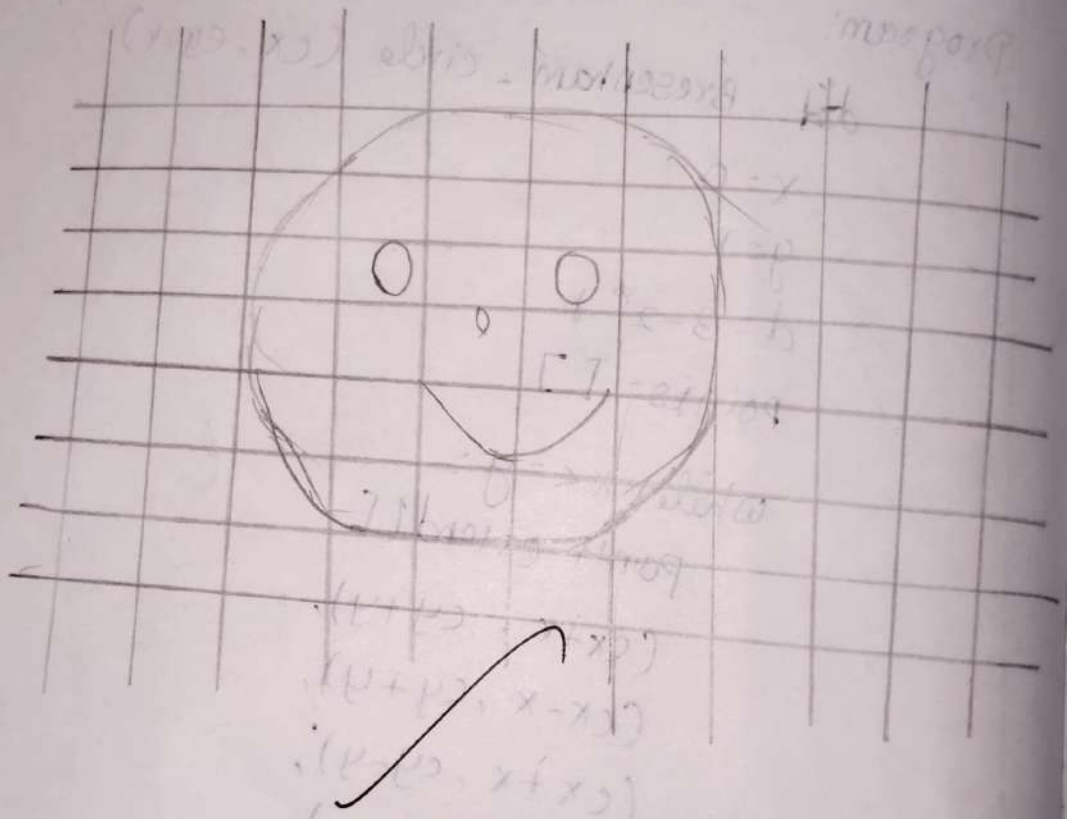
Smiley face using Bresenham's algorithm

AIM:

To Plot Smiley face using Bresenham algo.

Program:

```
def Bresenham_circle (cx, cy, r):
    x = 0
    y = r
    d = 3 - 2 * r
    points = []
    while x <= y:
        points.extend([
        (cx+x, cy+y),
        (cx-x, cy+y),
        (cx+x, cy-y),
        (cx-x, cy-y)
    ])
    if d < 0:
        d = d + 4 * x + 6
    else:
        d = d + 4 * (x-y) + 10
        y -= 1
    x += 1
    return points
def draw_circle (cx, cy, r):
    points = bresenham_circle (cx, cy, r)
    x = [p[0] for p in points]
    y = [p[1] for p in points]
```



```
plt.scatter(x, y, s=10, color="black")

draw_circle(250, 250, 150)
draw_circle(200, 300, 20)
draw_circle(300, 300, 20)

points = bresenham_circle(250, 250, 80)

smile_x = []
smile_y = []
for x, y in points:
    if y < 250:
        smile_x.append(x)
        smile_y.append(y)
plt.xlim(50, 450)
plt.ylim(50, 450)
plt.gca().set_aspect("equal")
plt.axis("off")
plt.title("smiling face")
plt.show()
```

Result:

Smiley face has been plotted using Bresenham's Algorithm.

22/09

EXP: 10

IMPLEMENT SCAN CONVERSION OF POLYGON & USE FLOOD FILL.

AIM:
TO implement scan conversion of Polygon and use flood fill algorithm.

Program:

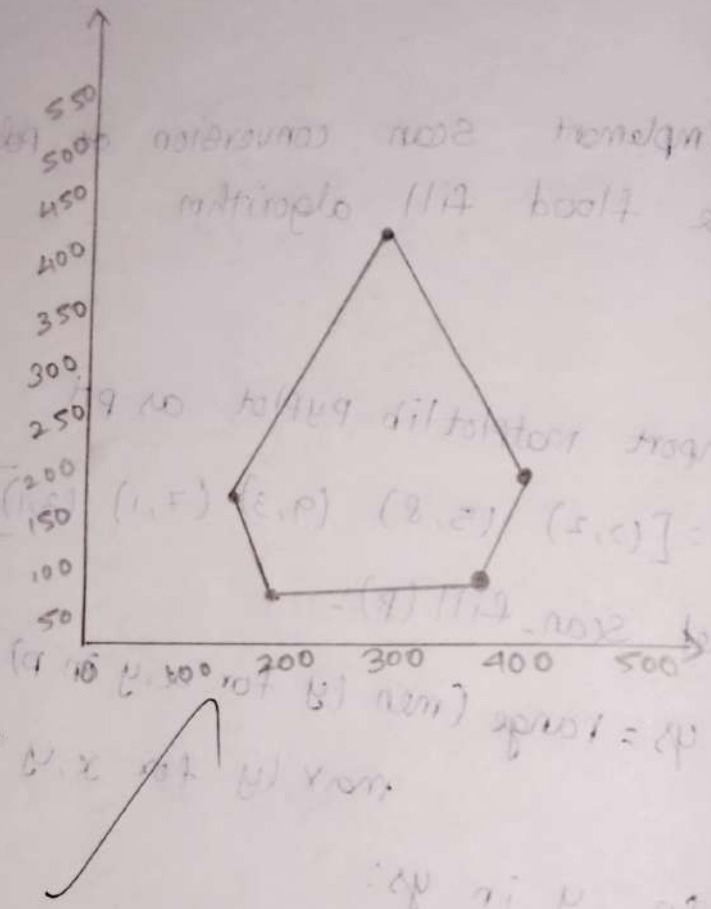
```
import matplotlib.pyplot as PH
P=[(2,2) (5,8) (9,3) (7,1) (3,1)]
def scan_fill(P):
    ys = range (min (y for x,y in P),
                max (y for x,y in P)+
    fo y in ys:
        x= []
        for i in range(len(P)):
            x1, y1 = P[i]
            x2, y2 = P[(i+1) %. len(P)]
            ff y1 != y2 and min (y1, y2)
            x.apped (x1 + (y-y1)*(x2-x1)
                     (y2-y1))
        x.sort()
    for i in range (0, len(x), 2):
        plt.plot [x[i:i+2], [y,y];
```

```
x, y = zip (* (p + [p[0]]))
plt.plot (x, y, 'k-')
scan_fill (p)
PH.fill (x, y, color='lightblue', alpha=0.5)
plt.axis ('equal')
PH.title ("scan conversion & flood fill")
plt.show()
```

22/7/20

Result:

Hence implemented scan conversion of polygon & used flood fill.


550
500
450
400
350
300
250
200
150
100
50
100
200
300
400
500

02/01

Exp: 11

2D Transformations in Triangle

AIM:

To implement Translation, Transformation and Scaling on triangle.

Program:-

```
import numpy as np
import matplotlib.pyplot as plt

original_shape = np.array([
    [0, 0],
    [2, 0],
    [1, 2],
    [0, 0]
])

def plot_shape(ax, shape, color, label):
    ax.plot(shape[:, 0], shape[:, 1], color=color,
            label=label, marker='o')
    ax.fill(shape[:, 0], shape[:, 1], color=color,
            alpha=0.3)
    ax.set_aspect('equal', adjustable='box')
    ax.set_xlim(-5, 5)
    ax.set_ylim(-5, 5)
    ax.grid(True)
    ax.axhline(0, color='black', linewidth=0.5)
    ax.axvline(0, color='black', linewidth=0.5)
```

```
def translate (shape, dx, dy):
    translation_vector = np.array([dx, dy])
    return shape + translation_vector

def rotate (shape, angle_degrees, pivot=(0,0)):
    angle_radians = np.deg2rad (angle_degrees)
    rotation_matrix = np.array([
    [np.cos(angle_radians), -np.sin(angle_radians)],
    [np.sin(angle_radians), np.cos(angle_radians)]
    ])
    translated_shape = shape - np.array(pivot)
    rotated_shape = translated_shape
    return rotated_shape + np.array(pivot)

    def scale (shape, sx, sy, pivot=(0,0)):
        scaled_shape = translated_shape
        translated_shape = shape - np.array(pivot)
```

1. Translation

```
translated_shape = translate (ori_shape, dx, dy)
plot_shape (ax, original_shape, 'blue')
ax.set_title ('2D Translation')
plt.show()
```

2. Rotation

```
rotated_shape = rotate (ori_shape, angle, pivot)
plot_shape (ax, ori_shape, angle)
ax.set_title ('2D rotation')
plt.show()
```

3. Scaling:

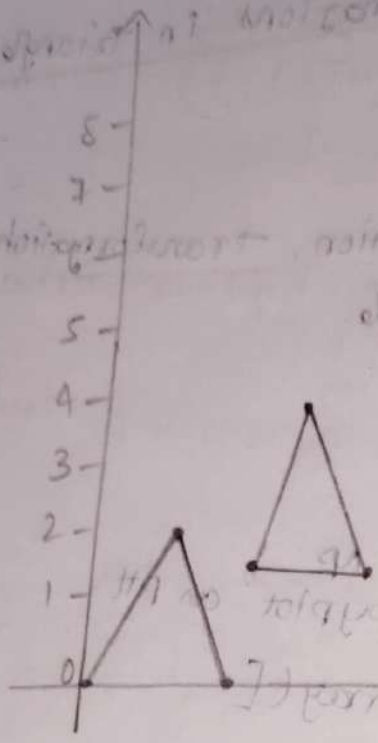
```
scald_shape = scale (ori_shape, sx, sy)
ax.set_title ('2D scaling')
plt.show()
```

28/9/26

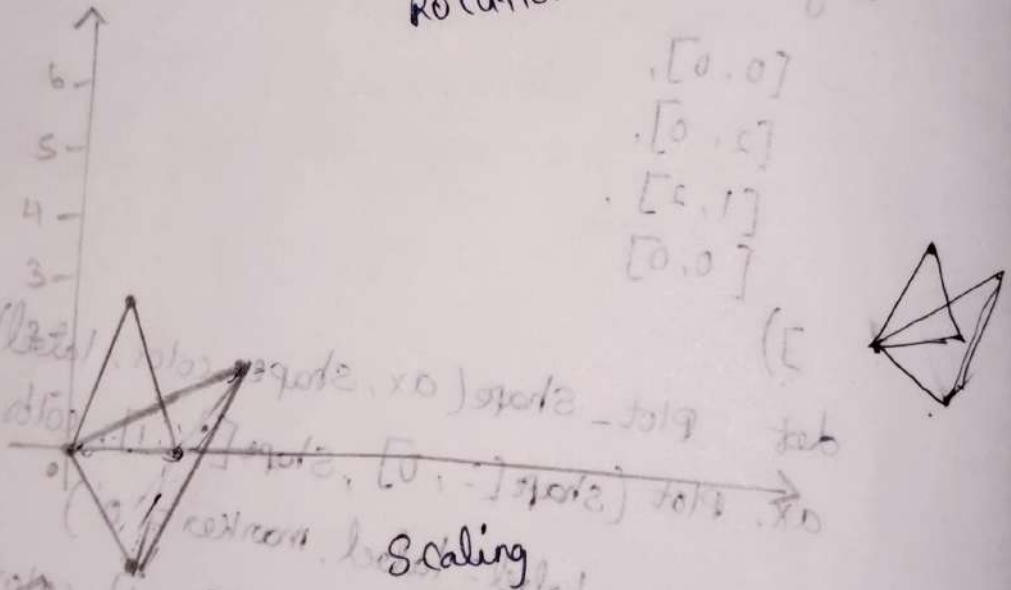
Result:

The program for 2D shape has been implemented.

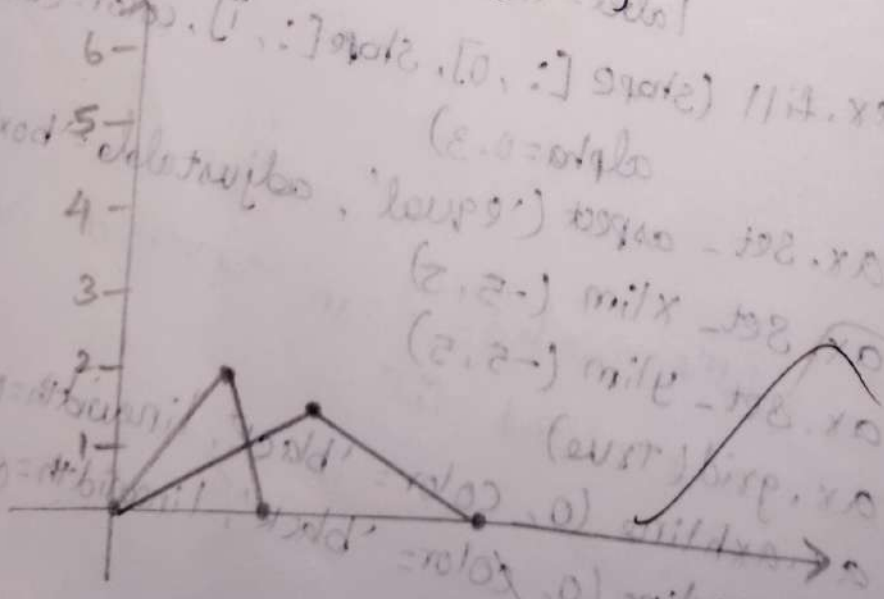
Translation



Rotation



Scaling



Clipping on polygon

AIM:-

TO implement clipping on polygon using sutherland hodgman

Program:

```
import numpy as np
import matplotlib.pyplot as plt

def inside (p, cp1, cp2):
    clip_edge_vec = cp2 - cp1
    point_vec = p - cp1
    return (clip_edge_vec[0] * point_vec[1])

def intersect (p1, p2, cp1, cp2):
    A1 = p2[1] - p1[1]
    B1 = p1[0] - p2[0]
    C1 = A1 * p1[0] + B1 * p1[1]

    A2 = [p2[1] - cp1[1]
    B2 = cp1[0] - cp2[0]
    C2 = A2 * cp1[0] + B2 * cp1[1]

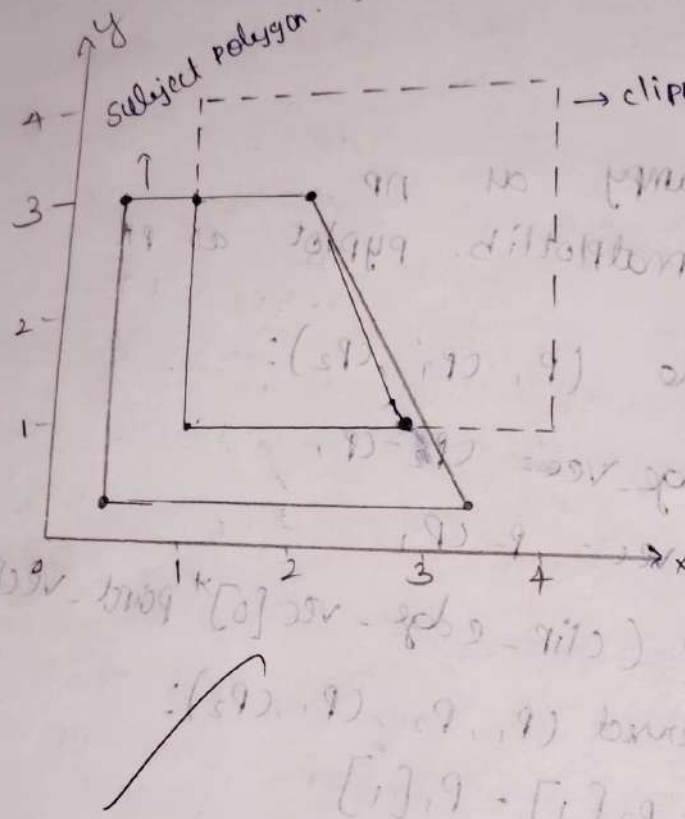
    if det == 0:
        return None
    else:
        x = (B2 * C1 - B1 * C2) / det
        y = (A1 * C2 - A2 * C1) / det
```

```
    return np.array([x,y])

def sutherland_hodgman(subject_polygon, clip_polygon):
    output_polygon = list(subject_polygon)
    for i in range(len(clip_polygon)-1):
        cP1 = clip_polygon[i]
        cP2 = clip_polygon[i+1]
        output_polygon = []
        if not input_polygon:
            break
        for j in range(len(input_polygon)):
            P1 = input_polygon[j-1]
            P2 = input_polygon[j]

        subject_vert = np.array([
            [0.5, 0.5],
            [3.0, 0.5],
            [2.0, 3.0],
            [0.5, 3.0]],
            [0.5, 0.5]
        ])
        clip_window = np.array([
            [1.0, 1.0]],
            [4.0, 1.0], [4.0, 4.0], [1.0, 1.0]
        ])
        ax.set_title('Sutherland hodgman polygon clipping')
        ax.set_xlabel('x-axis')
        ax.set_ylabel('y-axis')
        ax.legend()  plt.show()
```

has been implemented


y
subject polygon
clipping window
4
3
2
1
1
2
3
4
x

2D Transformations on ▭

AIM:

To implement 2D transformation in ▭.

Program:

```
original shape = np.array

dx, dy = 1, 2.5
fig, ax = plt.subplots (figsize = (8,8))
plot_shape (ax, ori_shape_rect)
plot_shape(ax, translated_rect)
ax.legend()
plot.show()

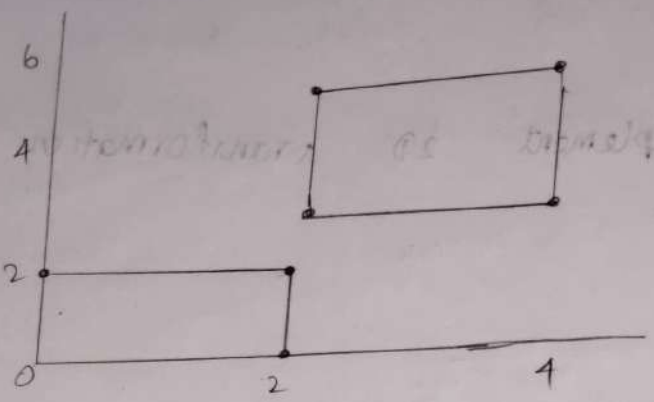
sx, sy = 0.5, 1.5
pivot_point = (0,0)
scaled_rect = scale (original_shape, sx, sy)
fig, ax = plt.subplots (figsize = (6,8))
ax.set_title ('2D Rotation (Rectangle)')
ax.legend()
plot.show()
scaled_rectangle = scale (ori_shape, sx, sy,
                          pivot = pivot)
fig, ax = plt.subplots (fg size-(6,6))
```

```
plot_shape (ax, original_shape, 'blue')
ax.set_title ('2D scaling (Rectangle)')
ax.legend()
plot.show()
```

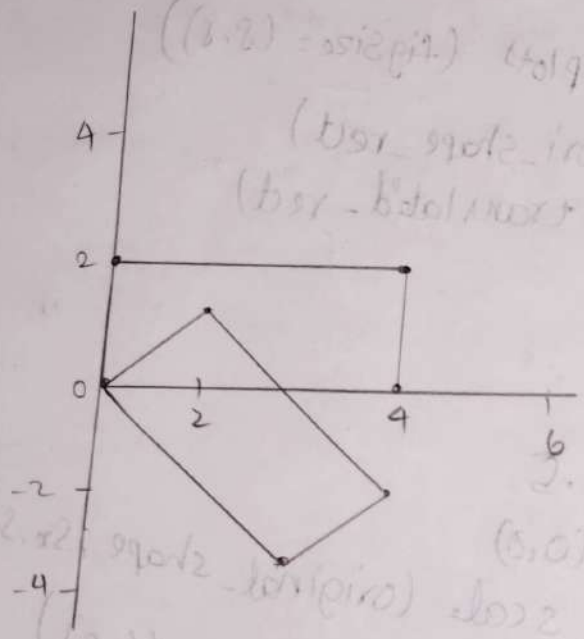
Result:

2D Transformations on rectangle has been implemented.

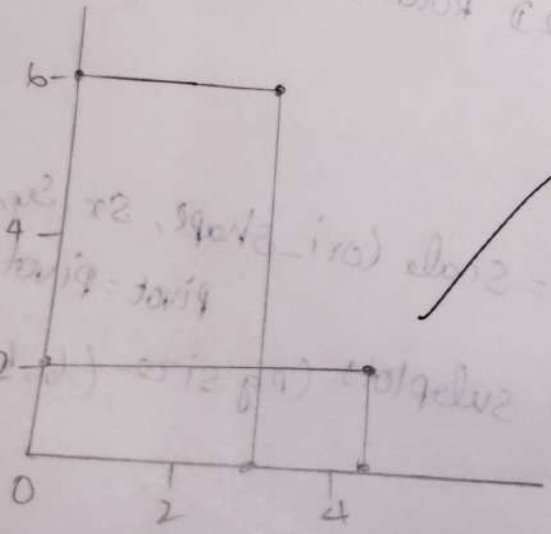
2D Translation



2D Rotation



2D Scaling



29/9/26

3D Translation on cube

AIM:
To implement transformations on cube.

Program:-

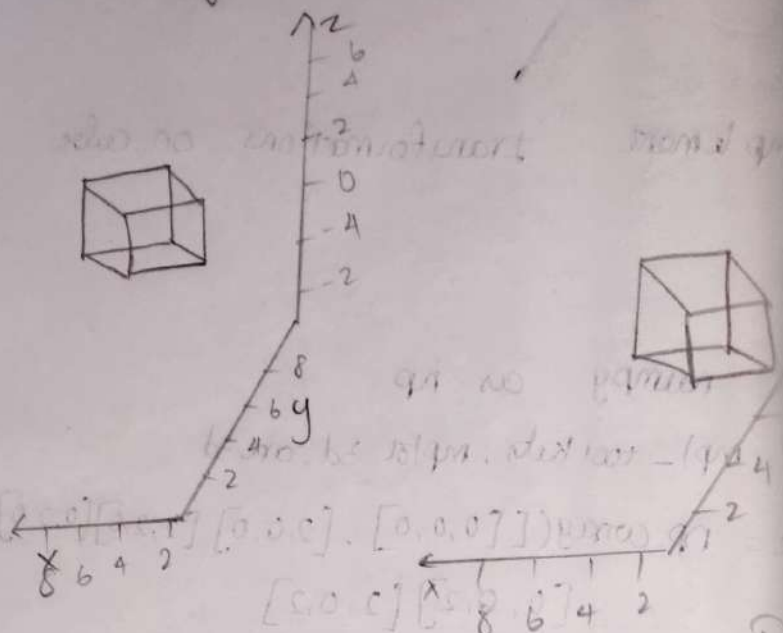
```
import numpy as np
from mpl_toolkits.mplot3d.art3d
cube = np.array([[0,0,0],[2,0,0][2,2,0][0,2,0],
                 [0,0,2][2,0,2]
faces= [[0,1,2,3] [4,5,6,7] [0,1,5,4],
        [2,3,7,6] [0,3,7,4]]
def transform (P, matrix):
    h = np.hstack((P, np.ones(len(P),1)))
    return (h @matrix, T)[:,:3]
T = np.array([[0,1,3,3][0,1,0,2][0,1,0,1]])
a = np.radians(45)
Rx = np.array([[1,0,0,0][0,np.cos(a),
                -np.sin(a),0],
Ry = np.array[ np.cos(a),0,np.sin(a),0],
     [0,1,0,0]
Rz = np.array([[np.cos(a),-np.sin(a),0,
                0]])
objects = [cube.transform(cube,T),
           transform(cube,S), (Rx,Ry,Rz)]
```

```
fig = plt.figure(figsize=(12,8))
for i, (obj, title) in enumerate(zip(objects, titles), 1):
    ax = fig.add_subplot(2,3,i, projection='3d')
    poly = [[obj][j] for j in f] for f in faces]
    ax.set_title(title)
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.set_zlabel("z")
    plt.tight_layout()
    plt.show()
```

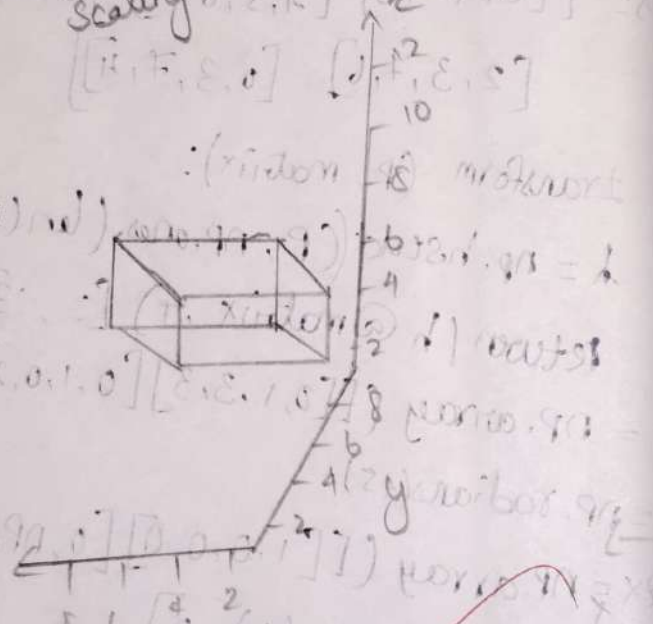
Result:

cube has been implemented using all transformations

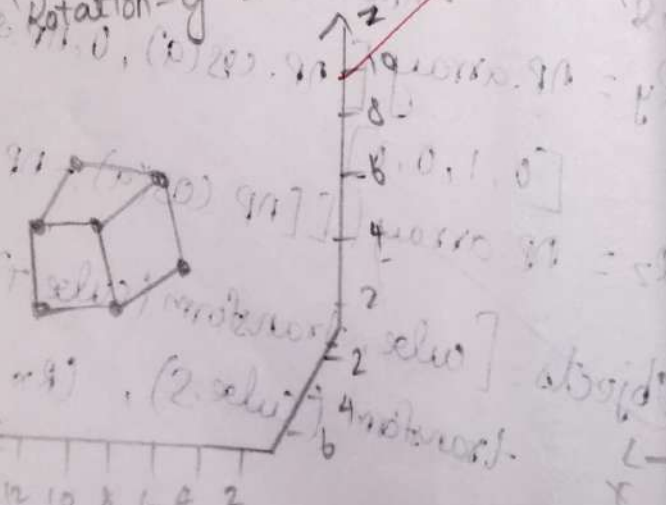
original axes



Scaling



Rotation - y



Transformation on 3D Pyramid

AIM:

To transform on pyramid shape with implementation

```
import numpy as np
import matplot.pyplot as plt

P= np.array([
    [0,0,0] [2,0,0] [2,2,0] ])
edges= [(0,1) (1,2)(2,3) (3,0)(0,4)(1,4) ]
def draw(ax, P, Style):
    ax.plot(*zip(P[a], P[b], Style)

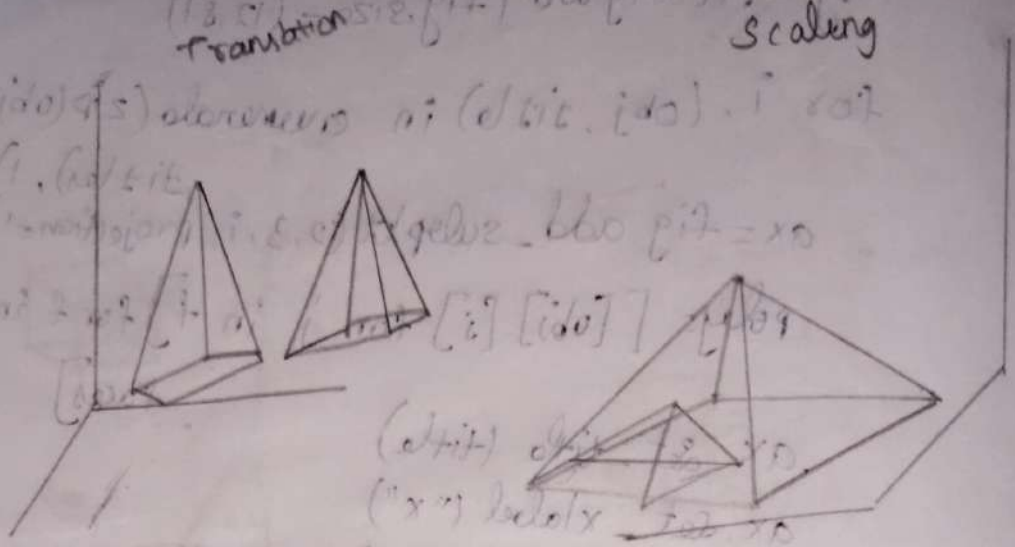
T= P+ [3,2,1]
S= P* (1.5, 1.5, 2]
theta= np.radians (45)
Rz= np.array([
    [0,0,1] ])
ax= fig.add_subplot (221, Prjctn = '3d')
ax_ set_title ("Translatn")
ax = fig.add_subplot (222, Prjctn = '3d')
ax_set title ("Scaling")
for ax in fig axes:
    ax.set_xlabel ("x")
    ax.set_ylabel ("y")
    ax.set_zlabel ("z")
plt.tight_layout()
PLt.show().
```

Result:

Pyramid has been implemented

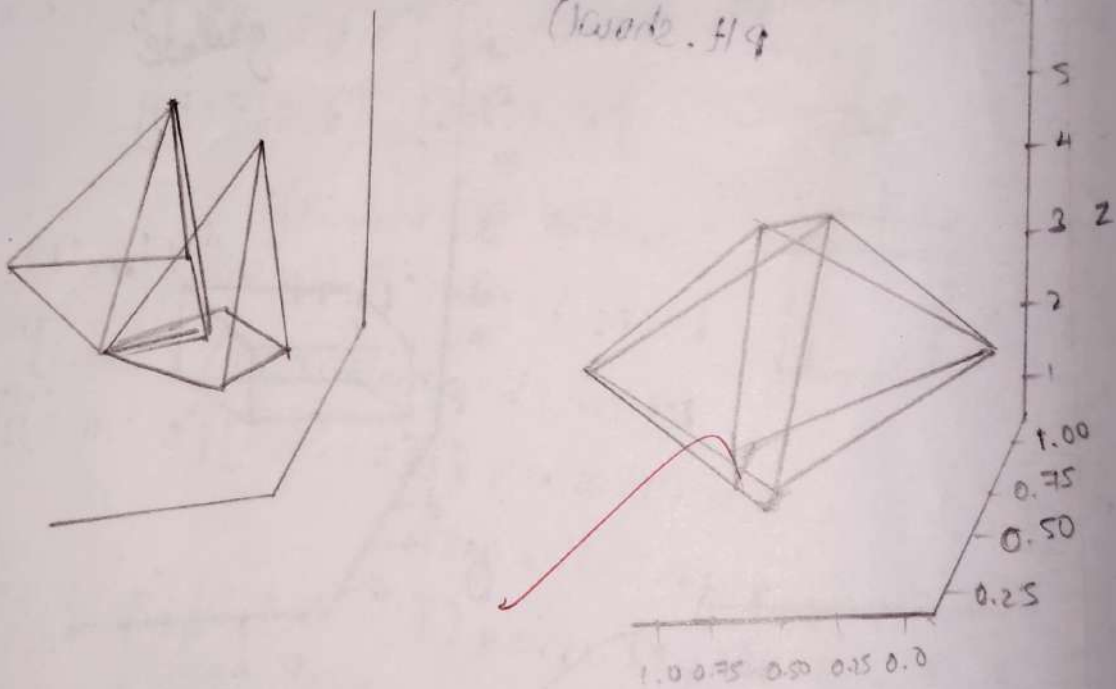
Translation

Scaling



Rotation about - z axis

Reflection in xy plane



Exp: 16 Create a multimedia application integrates images, sound and video.

AIM:

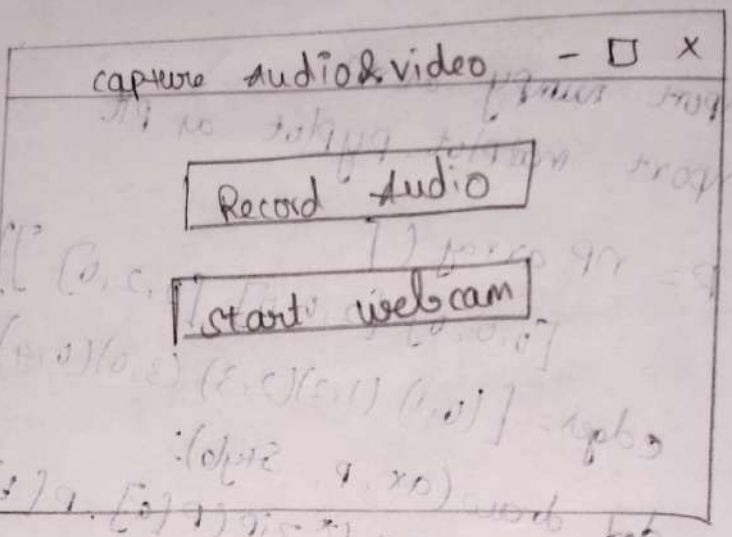
To develop a simple MM application.

Program:

```
import tkinter
From tkinter import filedialog
from PIL import image
From playsound import playsound
import CV2
import threading
def play-audio():
    Play sound()
def play-video():
    cap = CV2.video capture("Sample video")
while cap.isOpened():
        ref, frame
btki_audio = tk.bt(window_
    text = "play Audio", command = lambda:
btu_video = tk.button(window, text =
        "play")
Pand.pick()
btu_img.Pack(pady = 5)
window.mainlog()
```

Result:

Application has been implemented.


capture audio&video
Record audio
start webcam

- capture video & audio from webcam on microphone & display

AIM:

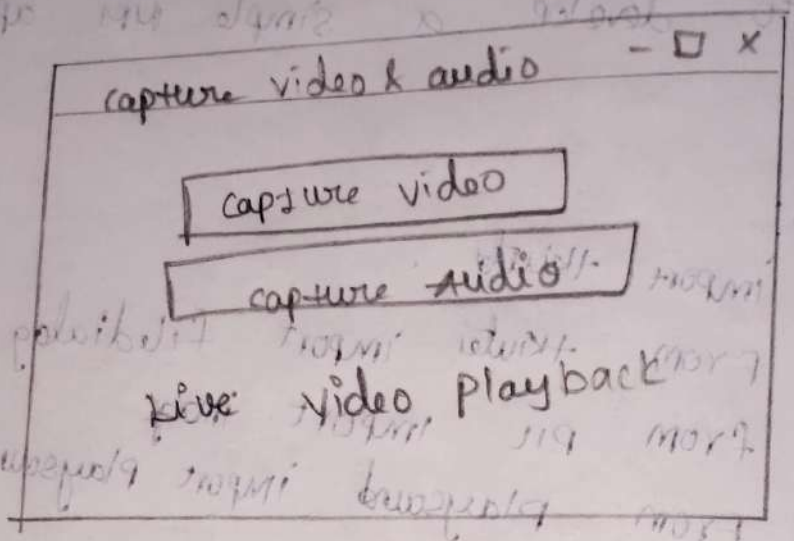
To dev a prgrm that captures live video from webcam.

Program:

```
import cv2
import sound devices
from scipy io.
import threading
import tkinter as tk
def_record_audio():
  duration = 5
  fs = 44100
  Print ("Recording audio")
  sd.wait()
  print ("Audio recording saved")
def capture_video():
    cp = cv2 - video capture()
    print (press 'q' to stop video')
  while True:
      ret, frame = cap.read()

  if not ret:
      break
  btu.audio.pack (pady = 10)
  btu.video.pack (pady = 10
  window.mainloop
```

Result: captured live video from webcam successfully


capture video & audio
capture video
capture audio
live video playback

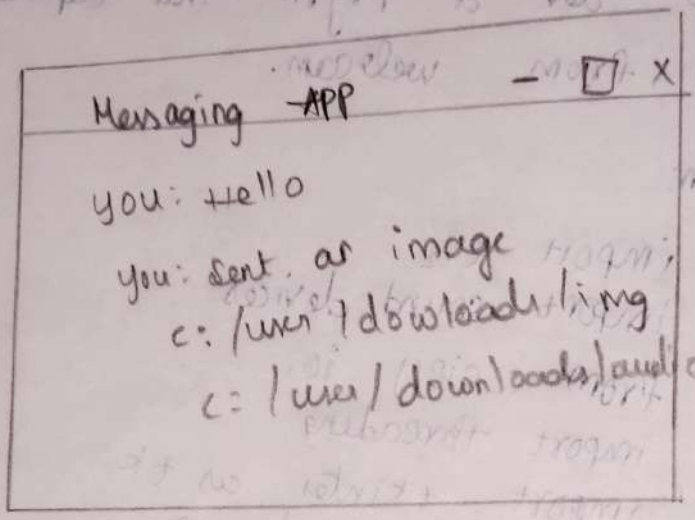
Create a Application to send and Receive MM Message

AIM:
To develop a basic chat-like application.

Program:

```
import tkinter as tk
from tkinter import file dialog
from PIL import image. image Tk
From playsound import
det send_text():
    msg = text_ Extra.get()
if msg:
    text_entry.delete
det send_image():
    btn.text.Path (Side = tk.left)
    btn.text.path (side = tk.left)
    btn.Audio.Path (side = tk.left)
    chat_box = Scrolled.text.
ScrolledText (chat, frame,
                wrap = tk.word,
                height = 20) width = 50,
chat_box.pack()
chat.frame.path
window.moving log()
```

Result:
developed a chat-like application.


Messaging APP
you: Hello
you: Sent as image
c:/user/downloads/img
c:/user/downloads/audio

EXP: 19

Create 3D using blender & apply basic shading texture

AIM: To create a simple 3D model using blender.

Procedure:

1. Launch blender
2. Model creation
   * use edit mode controlling & shading.
3. Shading
   * Switch to shading
   * Add a new material for object.
   * Adjust base color, Specular roughness.
4. Render and Mapping:
   * In the shader editor, add an image texture node.
   * Use UV editing to align the texture.
5. Lighting & camera
6. ~~PO~~ Settings

Result:

A 3D obj was modelled & verified successfully.